

Technical Verification Appendix

Finite Rational Reductions for the Convex-to-Polytope Conjecture in Neural Codes

Alec Kriebel

August 2026

1 Exact finite examples and verifier scope

The accompanying scripts use exact rational arithmetic. They check finite calibrations; they do not replace the universal proofs.

1.1 Rational interval events

The verifier represents interval endpoints by rational numbers and evaluates membership on every endpoint class and complementary open cell. It includes simultaneous starts and ends, empty intervals, full intervals, and marked endpoint words.

1.2 Fixed-arrangement cells

Small rational arrangements include a tangent cell, a lower-dimensional cell, a wholly outside atom, and a split atom with an outside witness on a separator boundary. Exact halfspace signs confirm the theorem’s classifications.

1.3 Sequential and synchronized repair

For the sequential-repair example in the main manuscript, exact polygon membership confirms:

1. the protected words 13 and 123 occur;
2. the first synchronized equality $P_1 = P_3$ removes word 1;
3. $(-1, -3/10)$ has word 1 after the later one-neuron hull;
4. $P_1 = P_3 = P^{\text{hull}}$ realizes exactly $\{13, 123\}$.

1.4 One-round failure

The optional Phase-XIX calibration verifies a three-neuron source code $\{\emptyset, 1, 2, 123\}$, a protected supercode with an unwanted 12 atom, a complete 123 repair patch covering the old carrier, and a remote rational 12 witness after the one-round synchronized hull. A second common-core polytope eliminates the defect. This example is included to falsify an overly strong globalization conjecture, not as a component of any universal proof.

1.5 Scope statement

The general theorems are human-readable mathematical proofs. The scripts certify only the stated finite coordinate examples, combinatorial deletion orders, and rational endpoint data. Failure of a finite search or a particular ansatz is nowhere used as evidence of nonpolytopality.

2 Verification specifications and proof-computation boundary

The reproducibility package accompanying this paper has two distinct roles. It is not a formal proof assistant for the universal statements. Instead, it provides deterministic exact-arithmetic replay for the finite coordinate examples and for combinatorial identities that are especially vulnerable to strict-versus-weak inequality mistakes. The general theorems are certified by the mathematical arguments in the main text.

2.1 Exact interval traces

A rational interval trace is encoded by a finite family of open intervals

$$J_j = (a_j, b_j) \subset T = (A, B)$$

with rational endpoints. The verifier forms the finite ordered partition of T determined by all endpoint equality classes. It evaluates the relative word on every complementary open interval and at every endpoint class. This catches the two errors that are invisible to generic sampling: a simultaneous start/end event that is accidentally ordered, and an endpoint word that changes when strict inequalities are replaced by weak ones. Rationalization is checked by comparing the complete ordered-partition word table before and after replacing the source endpoint values.

2.2 Finite arrangement separation

The fixed-arrangement calibration uses rational halfspaces and checks four cell types: a relatively open full-dimensional cell wholly outside the selector, a split cell, a lower-dimensional cell, and a tangent cell whose closure meets the selector. For each retained separator, the script verifies that every protected inside point satisfies the inequality strictly and that the designated outside witness violates it weakly. The finite intersection is then evaluated on one representative from every arrangement cell and every lower-dimensional face used in the example.

This computation illustrates, but does not replace, the fixed-arrangement proof in the main manuscript. In the theorem, finiteness comes from the old facet arrangement, and separation is invoked once for each wholly outside cell and once for a chosen outside witness in each split cell.

2.3 Repair-atlas calibrations

The planar sequential-repair example is checked using exact rational polygon membership. The script verifies the protected words 13 and 123, the disappearance of word 1 after the first synchronized equality $P_1 = P_3$, the reappearance of word 1 at

$$(-1, -3/10)$$

after a later one-neuron hull, and the exact final code $\{13, 123\}$ under the synchronized replacement $P_1 = P_3 = P^{\text{hull}}$.

The optional one-round calibration uses exact rational convex hulls and branchwise strict linear feasibility. It verifies a source code $\{\emptyset, 1, 2, 123\}$, an inner-polytopal supercode with one unwanted 12 carrier, a complete 123 atlas patch covering the old carrier, and the remote forbidden witness $(0, 1)$ after one-round synchronized hulls. A second common-core polytope is then checked to restore the exact target code. This example is included only to demonstrate that finite simultaneous attachment is nontrivial; no finite-stage failure is interpreted as a nonpolytopality obstruction.

2.4 Manifest and clean-build policy

The release contains a SHA-256 manifest over all source, verifier, metadata, and outreach files. The replay script first checks this manifest, runs the exact examples, compiles the manuscript and this appendix from source, and compares the resulting artifact hashes against the release ledger. The arXiv tarball is separately extracted and compiled in a clean directory with no network access and no private path dependencies.

The proof-computation division is therefore explicit:

Claim type	Certification method
Universal geometric theorem	Human-readable proof in the manuscript
Finite rational coordinate example	Deterministic exact arithmetic
Strict/weak endpoint table	Exhaustion of endpoint classes and open cells
Fixed finite LP branch	Rational feasibility witness or Motzkin/Farkas certificate
Absence of bounded-search output	No mathematical conclusion drawn

This separation prevents the verifier from being presented as evidence for claims outside its finite scope.